Universidade Federal do Rio Grande do Norte

Unidade Acadêmica Especializada em Ciências Agrárias

Escola Agrícola de Jundiaí

Curso de Análise e Desenvolvimento de Sistemas

Fundamentos e Técnicas em Ciência de Dados

ORQUESTRAÇÃO DE PROCESSOS COM APACHE AIRFLOW

Macaiba-RN

2026

Resumo

A orquestração de processos coordena atividades que precisam ser executadas de maneira organizada, respeitando dependências e condições de execução. Este relatório apresenta os conceitos fundamentais de orquestração e o Apache Airflow, plataforma de código aberto utilizada para definir, agendar e monitorar fluxos de trabalho. São abordados o histórico da ferramenta, o conceito de Directed Acyclic Graph (DAG), tarefas, operadores, componentes da arquitetura, execução, monitoramento e tratamento de falhas. Como aplicação prática, descreve-se uma prova de conceito (POC) de irrigação inteligente que simula a coleta de variáveis ambientais — umidade do ar, temperatura, umidade do solo e previsão de chuva — para avaliar a necessidade de irrigação. A POC demonstra como as etapas de um processo podem ser organizadas em uma DAG e como uma decisão condicional pode direcionar o fluxo. Por se tratar de uma simulação, os resultados representam o comportamento lógico do processo, não o acionamento físico de equipamentos agrícolas.

1 INTRODUÇÃO

Sistemas computacionais frequentemente dependem da execução coordenada de diversas atividades. Um processo pode envolver a obtenção de dados em diferentes fontes, validação, transformação, análise, armazenamento e comunicação dos resultados. Quando essas etapas são executadas isoladamente ou por meio de comandos manuais, torna-se mais difícil garantir a ordem correta, acompanhar falhas e manter a regularidade das operações.

Nesse contexto, a orquestração de processos organiza e coordena tarefas, estabelecendo dependências e condições para que um fluxo de trabalho seja executado de maneira previsível. Uma analogia é a de uma orquestra musical: os músicos representam atividades distintas, a partitura representa o fluxo planejado e o maestro coordena o momento e a sequência em que cada parte deve acontecer.

Uma ferramenta utilizada para esse propósito é o Apache Airflow. Trata-se de uma plataforma de código aberto na qual os fluxos de trabalho são definidos como código, normalmente em Python, e representados por grafos acíclicos direcionados, conhecidos como DAGs. A ferramenta permite definir tarefas, dependências e agendamentos, além de acompanhar as execuções por meio de uma interface.

Este trabalho apresenta os fundamentos da orquestração com Apache Airflow e descreve uma prova de conceito voltada à simulação de um sistema inteligente de irrigação. Nesse cenário, dados ambientais são analisados para apoiar a decisão de acionar ou não a irrigação. O foco da POC é demonstrar a coordenação das etapas, e não implementar um sistema agrícola físico completo.

2 OBJETIVOS

2.1 Objetivo geral

Compreender os princípios da orquestração de processos e demonstrar a utilização do Apache Airflow por meio de uma prova de conceito que organiza um fluxo simulado de decisão de irrigação.

2.2 Objetivos específicos

- Apresentar o conceito de orquestração e diferenciá-lo da execução isolada de tarefas.
- Descrever a origem, as características e as aplicações do Apache Airflow.
- Explicar DAG, Task, Operator, Scheduler, Executor e interface de monitoramento.
- Demonstrar como dependências e condições organizam um fluxo de trabalho.
- Modelar um processo de análise de variáveis ambientais para decidir sobre a necessidade de irrigação.
- Discutir os resultados, as limitações e as possibilidades de evolução da POC.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 Automação e orquestração

Automação consiste em executar uma atividade com pouca ou nenhuma intervenção manual. A orquestração, por sua vez, coordena várias atividades dentro de um fluxo maior. Assim, automatizar uma tarefa não significa necessariamente orquestrar um processo completo.

Por exemplo, um programa pode automatizar a leitura de um arquivo. Entretanto, se for necessário garantir que a leitura aconteça antes da transformação dos dados e que o armazenamento ocorra somente após a transformação, existe uma necessidade de coordenação. A orquestração define essas relações, controla condições de execução e oferece uma visão do processo como um todo.

Na analogia musical, cada músico executa uma parte, mas a música depende da organização conjunta. Na tecnologia, as tarefas são os participantes do processo, as dependências indicam a sequência e a ferramenta de orquestração coordena o fluxo.

3.2 História do Apache Airflow

O Apache Airflow teve início em outubro de 2014, quando Maxime Beauchemin começou a desenvolvê-lo no Airbnb para atender necessidades relacionadas à organização de fluxos de trabalho de dados. O projeto foi disponibilizado publicamente em 2015, passou a integrar o programa Apache Incubator em 2016 e foi reconhecido como projeto de nível superior da Apache Software Foundation em janeiro de 2019.

A proposta de definir workflows como código tornou possível descrever processos em Python, permitindo que fossem versionados, revisados e mantidos como parte de um projeto de software. A versão 2.0, lançada em 2020, representou uma etapa importante da evolução da plataforma, com melhorias de arquitetura e experiência de utilização. Por ser um projeto de código aberto, o Airflow conta com uma comunidade que contribui com correções, documentação e integrações .

3.3 DAG, Task e Operator

A DAG (Directed Acyclic Graph), ou grafo acíclico direcionado, representa um workflow. Directed indica que as relações possuem direção; Acyclic significa que não há ciclos no grafo; e Graph representa o conjunto de nós e conexões. No Airflow, os nós representam tarefas e as conexões indicam suas dependências.

Uma Task é uma unidade de trabalho dentro da DAG. Ela pode representar uma consulta a uma API, validação de dados, execução de um script ou envio de uma notificação. Um Operator é uma abstração utilizada para definir o tipo de operação executada por uma tarefa. Há operadores para executar código Python, comandos de terminal e interações com serviços externos.

3.4 Componentes da arquitetura

O Airflow reúne componentes que interpretam as definições das DAGs, decidem quando executar tarefas, encaminham atividades para execução e registram seus estados. A arquitetura pode variar conforme a versão e a forma de implantação. Em uma visão conceitual, destacam-se:

- Scheduler: acompanha as DAGs e identifica quando um fluxo ou tarefa está apto a ser executado, respeitando o agendamento e as dependências.
- Executor: define como e onde as tarefas serão executadas. O tipo configurado influencia a execução local, paralela ou distribuída.
- Interface web: permite consultar DAGs, execuções, estados e logs. Nas versões atuais, a arquitetura utiliza um API Server para disponibilizar a interface e a API.
- Metadata Database: armazena metadados sobre DAGs, tarefas, execuções e estados.
- Processamento de DAGs: interpreta os arquivos que definem os workflows e disponibiliza suas estruturas para a plataforma.

Os nomes e a organização dos componentes podem variar entre versões. A documentação oficial apresenta a arquitetura e as opções de implantação em maior detalhe [2].

3.5 Agendamento, dependências e execução paralela

Uma DAG pode ser executada manualmente ou de acordo com um agendamento. O Scheduler verifica se chegou o momento de iniciar uma execução e se as condições necessárias foram atendidas. Dentro da DAG, as dependências impedem que uma tarefa seja iniciada antes das etapas das quais depende.

Quando duas tarefas não possuem dependência entre si, elas podem ser executadas em paralelo, desde que a configuração do ambiente e os limites de concorrência permitam. Isso pode reduzir o tempo total de processos com etapas independentes, mas exige atenção à capacidade de processamento e ao compartilhamento de dados.

3.6 Monitoramento e tratamento de falhas

A interface do Airflow permite acompanhar o estado das execuções. Os logs registram mensagens produzidas durante o processamento e ajudam a identificar em qual etapa ocorreu um problema. Também é possível configurar novas tentativas (retries), intervalos entre tentativas e limites de tempo.

Repetir uma tarefa sem considerar seus efeitos pode causar duplicidade ou inconsistência. Por isso, recomenda-se projetar tarefas idempotentes sempre que possível, isto é, tarefas cuja repetição não produza efeitos indevidos. Para troca de informações, o Airflow oferece recursos como XCom para pequenos valores; arquivos e grandes volumes devem preferencialmente ser armazenados em locais compartilhados, transmitindo-se entre tarefas apenas suas referências [3].

3.7 Pipelines de dados

Um pipeline de dados é uma sequência organizada de etapas que movimenta e transforma informações. Em um processo ETL (Extract, Transform, Load), os dados são extraídos de fontes, transformados e carregados em um destino. No ELT (Extract, Load, Transform), os dados são carregados antes de serem transformados no ambiente de destino.

O Airflow pode orquestrar esses fluxos ao coordenar consultas, transformações, validações e carregamentos. A ferramenta não substitui necessariamente os mecanismos especializados de processamento ou armazenamento: ela organiza a execução das etapas e acompanha seus resultados. O mesmo princípio pode ser aplicado a pipelines de Machine Learning, relatórios e integração de sistemas.

3.8 Vantagens e limitações

Vantagens

- Workflows definidos como código, facilitando versionamento e revisão.
- Visualização centralizada de DAGs, tarefas e histórico de execução.
- Agendamento e controle de dependências.
- Possibilidade de execução paralela e integração com serviços externos.
- Recursos de logs, tentativas e monitoramento.

Limitações e cuidados

- Exige configuração e manutenção do ambiente, especialmente em implantações distribuídas.
- A lógica de negócio precisa ser implementada nas tarefas ou em sistemas externos; o Airflow não a cria automaticamente.
- Não é, em geral, a melhor opção para controle de eventos que exigem resposta em tempo real ou latência de milissegundos.
- Dependências, tentativas e compartilhamento de dados mal projetados podem gerar execuções ineficientes ou resultados inconsistentes.

O Airflow é especialmente adequado para workflows agendados ou disparados por eventos que precisam de coordenação e monitoramento. Em aplicações de IoT com requisitos estritos de tempo real, podem ser necessárias tecnologias especializadas em mensageria e processamento de eventos, enquanto o Airflow pode atuar em rotinas periódicas e análises.

4 METODOLOGIA E DESENVOLVIMENTO DA POC

4.1 Cenário proposto

A prova de conceito foi concebida como uma simulação de um sistema de apoio à decisão de irrigação. O cenário considera uma plantação monitorada por variáveis ambientais. A partir dessas informações, o processo avalia se as condições indicam necessidade de irrigação e segue um dos caminhos possíveis: simular o acionamento ou manter o sistema desligado.

O propósito principal da POC é demonstrar a orquestração de etapas com o Apache Airflow. Portanto, a coleta e a ação final são tratadas no contexto de simulação, não

sendo afirmada a existência de sensores físicos, conexão com uma estação meteorológica real ou acionamento de válvulas e bombas.

4.2 Variáveis consideradas

Variável	O que representa	Contribuição para a decisão
Umidade do ar	Quantidade de vapor de água no ar.	Contextualiza as condições atmosféricas.
Temperatura	Condição térmica do ambiente.	Pode influenciar a perda de água por evaporação e transpiração.
Umidade do solo	Disponibilidade de água no solo.	Indicador importante da necessidade hídrica.
Previsão de chuva	Estimativa de precipitação futura.	Pode evitar irrigação desnecessária quando há chuva prevista.

Em uma aplicação agrícola real, os limites de decisão não devem ser universais: precisam considerar a cultura, o estágio de desenvolvimento, o tipo de solo, a profundidade das raízes, as condições locais e a calibração dos sensores.

4.3 Organização do fluxo

O processo foi dividido em etapas com responsabilidades específicas. A estrutura apresentada na POC contempla a coleta das informações, a validação, a consolidação ou preparação dos dados, a análise das condições e a decisão sobre a irrigação. As etapas são conectadas por dependências dentro da DAG.

Etapa	Responsabilidade	Resultado esperado
Coleta	Obter ou simular os valores ambientais.	Conjunto inicial de variáveis.
Validação	Verificar se os dados necessários estão disponíveis e utilizáveis.	Dados aptos para análise ou identificação de inconsistências.
Consolidação	Organizar as informações em uma estrutura comum.	Dados preparados para análise.
Análise e decisão	Aplicar as regras definidas para avaliar a necessidade de irrigação.	Escolha do ramo: irrigar ou não irrigar.
Ação simulada	Executar o caminho correspondente à decisão.	Irrigação simulada como acionada ou mantida desligada.

4.4 Lógica de decisão

Após a preparação dos dados, o fluxo avalia as condições ambientais de acordo com as regras implementadas na POC. Quando a análise identifica necessidade de irrigação, o processo segue para a etapa que representa o acionamento. Caso contrário, segue para a etapa que representa a manutenção do sistema desligado. Essa separação caracteriza um fluxo condicional: nem todas as tarefas posteriores precisam ser executadas em todos os cenários.

No Airflow, esse comportamento pode ser implementado com recursos de controle de fluxo, como branching. Os critérios numéricos devem corresponder ao código efetivamente desenvolvido; valores ilustrativos usados em slides não devem ser interpretados como recomendações agronômicas.

5 RESULTADOS E DISCUSSÃO

A POC permite observar como um processo com diferentes etapas pode ser representado por uma DAG e acompanhado na interface do Apache Airflow. A visualização do grafo facilita a compreensão das dependências, enquanto o estado das tarefas permite verificar quais atividades foram executadas, quais foram ignoradas em função de uma condição e se houve falhas.

No cenário em que os dados indicam necessidade de irrigação, espera-se que o fluxo percorra o ramo associado ao acionamento simulado. No cenário oposto, deve seguir o ramo que mantém a irrigação desligada. A comparação evidencia que a orquestração não se limita a executar uma lista fixa de comandos: ela também pode organizar caminhos diferentes conforme os resultados produzidos pelas tarefas.

A demonstração destaca ainda a importância do monitoramento. Em um processo automatizado, não basta obter um resultado; é necessário identificar em qual etapa ele foi produzido e investigar eventuais falhas. Como a POC é simulada, não é possível concluir que houve economia real de água, aumento de produtividade ou melhoria efetiva da irrigação. Esses benefícios dependeriam de testes de campo, sensores calibrados, regras agronômicas validadas e integração segura com equipamentos.

6 POSSIBILIDADES DE EVOLUÇÃO

Uma evolução possível seria substituir os valores simulados por leituras de sensores reais, recebidas por uma API, gateway IoT ou plataforma de mensageria. Também seria possível integrar uma fonte confiável de previsão meteorológica e armazenar as leituras históricas em um banco de dados para análise posterior.

Outra melhoria seria registrar cada decisão com data, hora, variáveis observadas, regra aplicada e resultado. Esse histórico permitiria acompanhar a umidade do solo, comparar decisões e produzir relatórios. Alertas poderiam informar responsáveis quando os dados estivessem ausentes, fora de faixa ou quando uma tarefa falhasse.

Em uma solução real, o acionamento de bombas e válvulas exigiria integração específica com controladores ou dispositivos IoT, autenticação, proteção contra comandos duplicados e mecanismos de segurança. Também seria necessário prever falhas de comunicação, sensores defeituosos e falta de energia. O Airflow poderia coordenar rotinas periódicas de análise e relatórios, enquanto um componente especializado ficaria responsável pelo controle de baixa latência dos dispositivos.

7 CONCLUSÃO

A realização deste trabalho permitiu compreender a importância da orquestração na coordenação de processos compostos por várias tarefas. A analogia com uma orquestra evidencia que atividades individuais precisam seguir uma organização comum para que o resultado final seja alcançado de maneira consistente.

O Apache Airflow oferece recursos para representar workflows como código, organizar tarefas em DAGs, estabelecer dependências, agendar execuções e monitorar resultados. Sua utilização é relevante em cenários como engenharia de dados, integração de sistemas, automação de rotinas e pipelines de Machine Learning.

Na prova de conceito de irrigação inteligente, os conceitos foram relacionados a um cenário prático. A coleta, validação, consolidação e análise das variáveis ambientais foram organizadas como etapas de um fluxo, cuja decisão conduz a caminhos diferentes: simular o acionamento da irrigação ou mantê-la desligada. Dessa forma, a POC demonstra o papel do Airflow como coordenador do processo, sem confundi-lo com o sistema físico de sensores ou com o controlador do equipamento.

Conclui-se que a principal contribuição do Apache Airflow está em tornar processos complexos mais organizados, observáveis e gerenciáveis. A experiência também evidencia a necessidade de escolher a ferramenta adequada ao problema: para fluxos periódicos e coordenados, o Airflow é uma opção relevante; para controle de dispositivos em tempo real, sua utilização deve ser complementada por tecnologias específicas.

REFERÊNCIAS

[1] APACHE AIRFLOW. Project: History. Apache Airflow Documentation. Disponível em: https://github.com/apache/airflow/blob/main/airflow-core/docs/project.rst. Acesso em: 4 nov. 2026.

[2] APACHE AIRFLOW. Core Concepts. Apache Airflow Documentation. Disponível em: https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/. Acesso em: 4 nov. 2026.

[3] APACHE AIRFLOW. Best Practices. Apache Airflow Documentation. Disponível em: https://airflow.apache.org/docs/apache-airflow/stable/best-practices.html. Acesso em: 4 nov. 2026.

[4] APACHE SOFTWARE FOUNDATION. Apache Airflow. Disponível em: https://airflow.apache.org/. Acesso em: 4 nov. 2026.